11. gyakorlat

Python kurzus

Big Data II.

Adatvizualizáció

Machine Learning

Pandas Dataframe további utasítások:

Információs utasítások a df dataframe-ről (A SF_crime feladatsor példáival):

df.columns # Index(['IncidntNum', 'Category', 'Descript', 'DayOfWeek', 'Date', 'Time', ...], dtype='object')
len(df) # A df-ben lévő rekordok száma
df.isnull().sum() # Kategóriánként a NaN-ok száma jelenik meg
df.isnull().sum().sum() # A teljes df-ben a NaN értékek összes száma: np.int64(0) formában jelenik meg
print(df['Month'][0:5]) # Az első 5 érték megjelenítése az adott oszlopból
print(type(df['Month'][0])) # Az adatoszlopban lévő értékek típusának kiíratása

Adattisztítási lépések a df dataframe-ben:

df["PdDistrict"].fillna(df["PdDistrict"].mode()[0],inplace=True) # A NaN értékek helyére a móduszt írjuk
del df['IncidntNum'] # Az adott oszlop törlése a df-ből
df.drop('Location', axis=1, inplace=True) # Az adott oszlop törlése a df-ből a drop() metódussal

Adatelőkészítés lépései a df dataframe-ben:

df['Month'] = df['Date'].apply(lambda row: int(row[0:2])) # Új oszlop létrehozása meglévő oszlop értékeiből

Adatelemzési lépések a df dataframe-ben:

1. Összegzések, csoportosítások:

CountCategory = df['Category'].value_counts()
print(CountCategory) →

Új dataframe létrehozása összegzéssel a 'Category' oszlop értékei szerint.
A rendezés alapbeállítása csökkenő.

```
Category
LARCENY/THEFT          40409
OTHER OFFENSES         19599
NON-CRIMINAL           17866
ASSAULT                13577
VANDALISM               8589
                ...
Name: count, Length: 39, dtype: int64
```

print(df['PdDistrict'].value_counts(ascending=True)) →

Összegzés és növekvőre állított rendezés.

```
PdDistrict
PARK            8699
RICHMOND        8922
TENDERLOIN      9942
TARAVAL        11325
                ...
Name: count, dtype: int64
```

2. Szűrések:

JulyCrimes = df[df['Month'] == 7] # Új dataframe létrehozása csak a júliusi esetek szűrésével.

JulyVandalism = JulyCrimes[JulyCrimes['Category'] == 'VANDALISM'] # További adatszűrés

Crime0715 = df.query('Month == 7 and Day == 15') # A query alkalmazása logikai függvényekkel

Adatelemzési algoritmusok Python-ban, példák

1. példa Korrelációs analízis a pandas-sal:

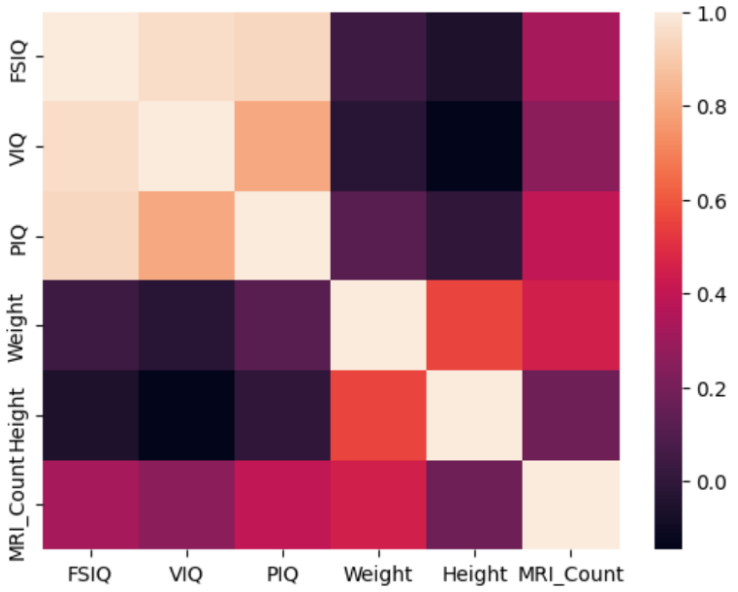
Az IQ és a fizikai adottságok kapcsolatának bemutatása
(ipynb munkafüzet, adatok: brainsize.txt)

Korrelációs mátrix:

```
brainFr_num.corr(method='pearson')
```

	FSIQ	VIQ	PIQ	Weight	Height	MRI_Count
FSIQ	1.000000	0.946639	0.934125	-0.051483	-0.086002	0.357641
VIQ	0.946639	1.000000	0.778135	-0.076088	-0.071068	0.337478
PIQ	0.934125	0.778135	1.000000	0.002512	-0.076723	0.386817
Weight	-0.051483	-0.076088	0.002512	1.000000	0.699614	0.513378
Height	-0.086002	-0.071068	-0.076723	0.699614	1.000000	0.601712
MRI_Count	0.357641	0.337478	0.386817	0.513378	0.601712	1.000000

A korreláció megjelenítése: seaborn heatmap-en



2. példa Döntési fa a Scikit-learn könyvtárral:

A Titanic utasainak adataiból Machine Learning algoritmussal levont következtetések bemutatása
Munkafüzet: Titanic_DontesiFa_Gyak.ipynb, az adatok: train.csv

A feladatban alkalmazott könyvtárak és osztályok importálása:

pandas, numpy, az sklearn.model_selection-ből a train_test_split, az sklearn.tree-ből a DecisionTreeClassifier, az sklearn.metrics-ből az accuracy_score, classification_report, confusion_matrix, az sklearn.preprocessing-ből a LabelEncoder, az sklearn.tree-ből a plot_tree és végül a matplotlib.pyplot

Az adatok felosztása tanuló (train) és teszt (test) halmazokra:

```
x_tanuló, x_teszt, y_tanuló, y_teszt = train_test_split(x, y, test_size=0.2, random_state=42)
```

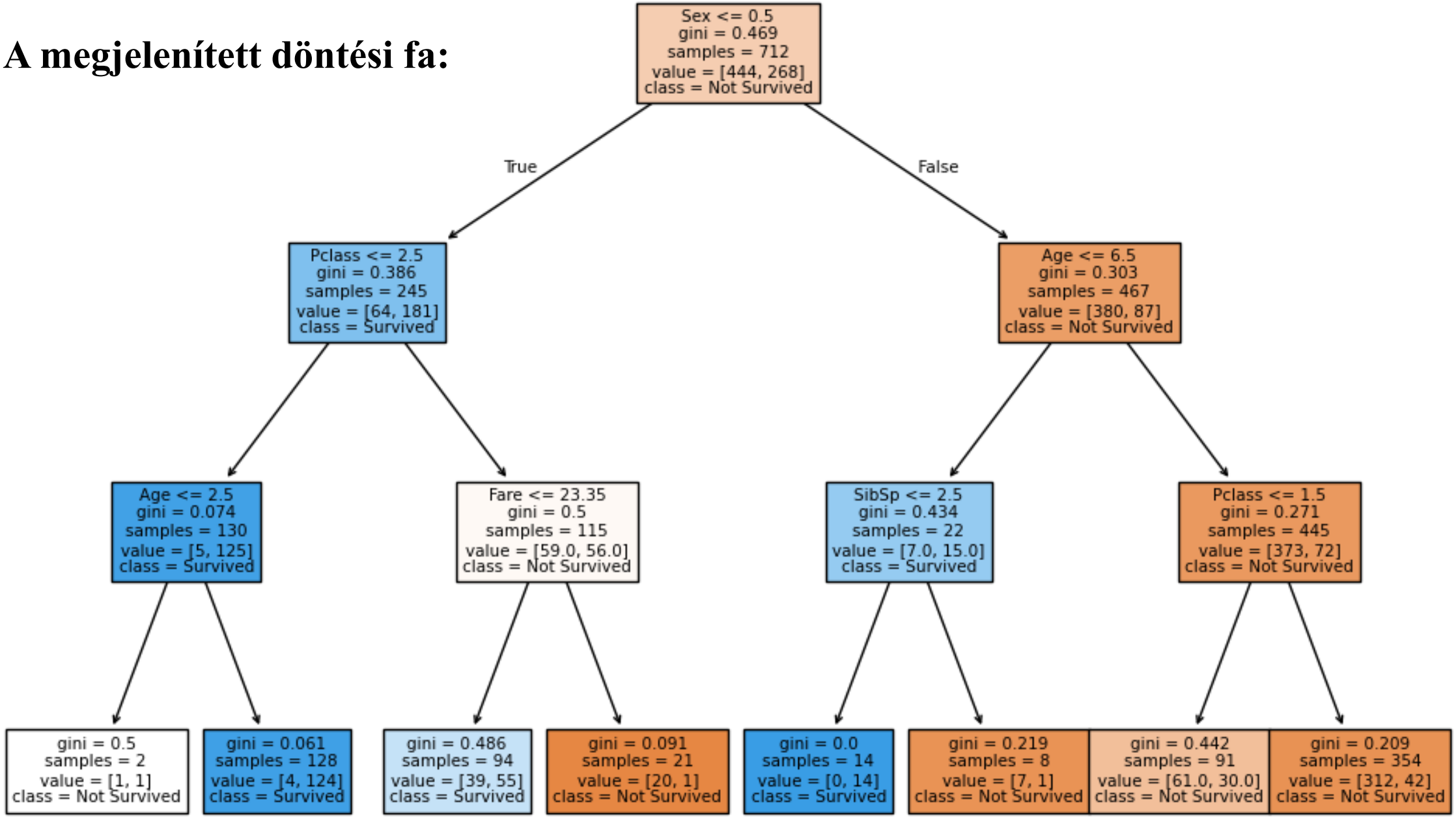
Modellépítés és betanítás:

```
dontesi_fa = DecisionTreeClassifier(max_depth=3, random_state=42)
dontesi_fa.fit(x_tanuló, y_tanuló)
```

A döntési fa vizualizációja a Matplotlib könyvtár segítségével:

```
plt.figure(figsize=(12, 8))
plot_tree(dontesi_fa, feature_names=x.columns, class_names=["Not Survived", "Survived"], filled=True)
plt.show()
```

A megjelenített döntési fa:



Adatvizualizáció

1. Matplotlib könyvtár

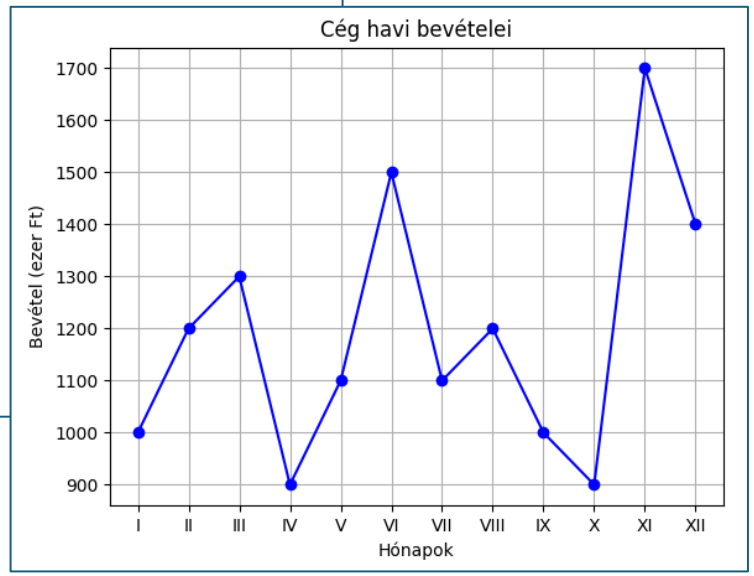
- A Matplotlib az adatvizualizáció egyik eszköztára
- Integrálható a Pandas és a NumPy adatfeldolgozó könyvtárakkal
- Telepítés: **pip install matplotlib**
- Vonaldiagramok, hisztogramok, scatter plotok és más diagramok készítésére: **plt.plot(), plt.subplot(1, 2, 1), plt.scatter(), plt.hist(), plt.boxplot(), plt.subplots()** és **twinx()**

Vonaldiagram

1. példa: Egy cég havi bevételeit ábrázoljuk vonaldiagramon.

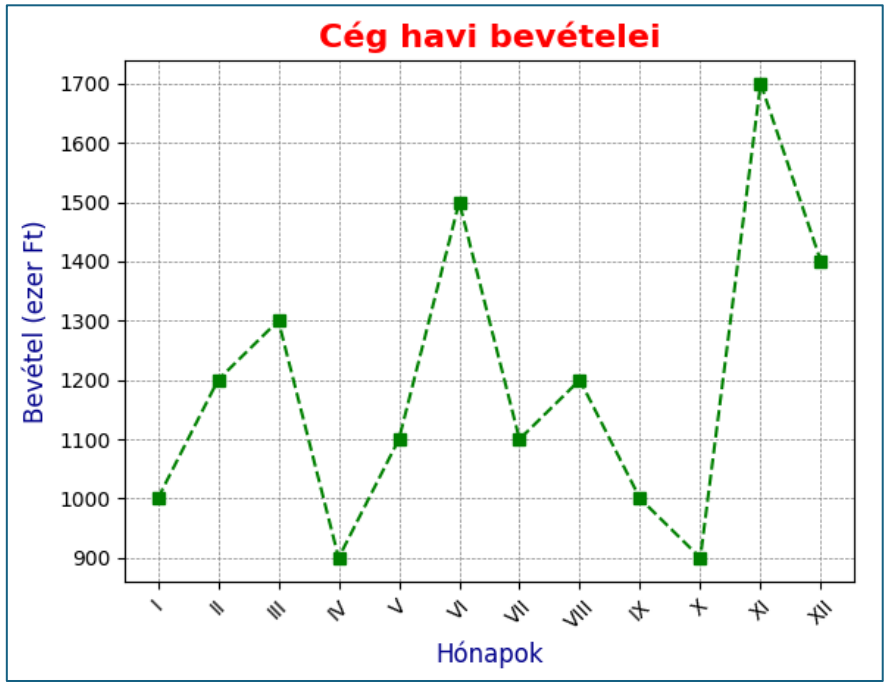
A **plt.plot()** függvényt alkalmazzuk a diagram létrehozásához.
Tengelyfeliratozás és -cím hozzáadása: **plt.xlabel(), plt.ylabel(), plt.title()**
A **marker** argumentum a pontok kiemelésére szolgál, a **linestyle** a vonal típusát állítja be. A diagram megjelenítés a **plt.show()**-val történik.

```
import matplotlib.pyplot as plt
# Adatok: hónapok és bevételek
hónapok = ['I', 'II', 'III', 'IV', 'V', 'VI', 'VII', 'VIII', 'IX', 'X', 'XI', 'XII']
bevételek = [1000, 1200, 1300, 900, 1100, 1500, 1100, 1200, 1000, 900, 1700, 1400]
# Vonaldiagram készítése
plt.plot(hónapok, bevételek, marker='o', linestyle='-', color='b')
plt.title('Cég havi bevételei')
plt.xlabel('Hónapok')
plt.ylabel('Bevétel (ezer Ft)')
plt.grid(True)
plt.show()
```



Tengelyek, címek és feliratok testreszabása

```
plt.plot(hónapok, bevételek, marker='s', linestyle='--', color='g')
plt.title('Cég havi bevételei', fontsize=16, fontweight='bold', color='navy')
plt.xlabel('Hónapok', fontsize=12, color='darkgreen')
plt.ylabel('Bevétel (ezer Ft)', fontsize=12, color='darkgreen')
plt.xticks(rotation=45)
plt.grid(color='gray', linestyle='--', linewidth=0.5)
plt.show()
```



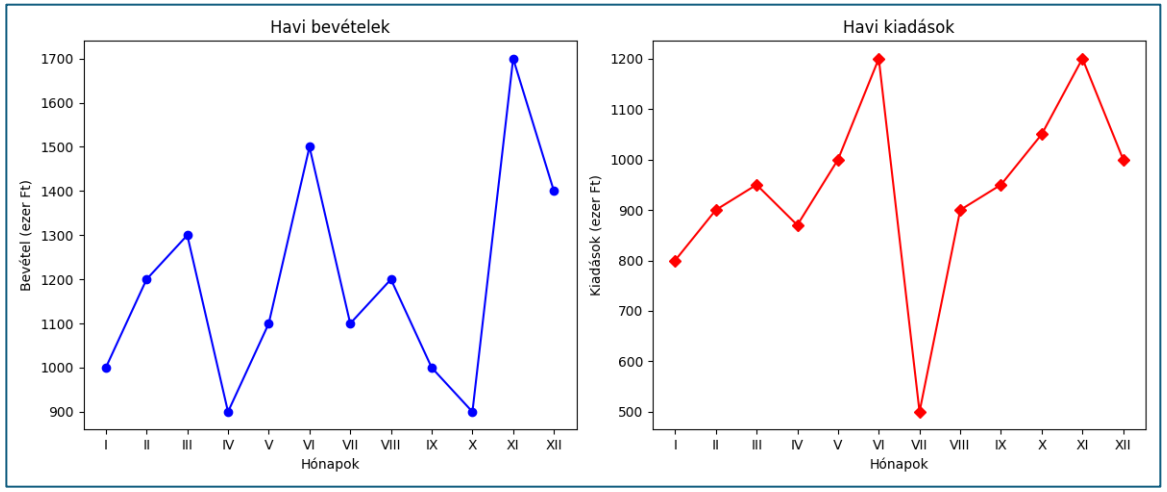
Cím és tengelyek formázása: betűméret, szín, stílus beállítása.
Rotáció alkalmazása az x tengelyen: **plt.xticks(rotation=45)**
Rácsvonalak vezérlése: **plt.grid()**

Többszörös diagramok egy ábrán belül - subplotok

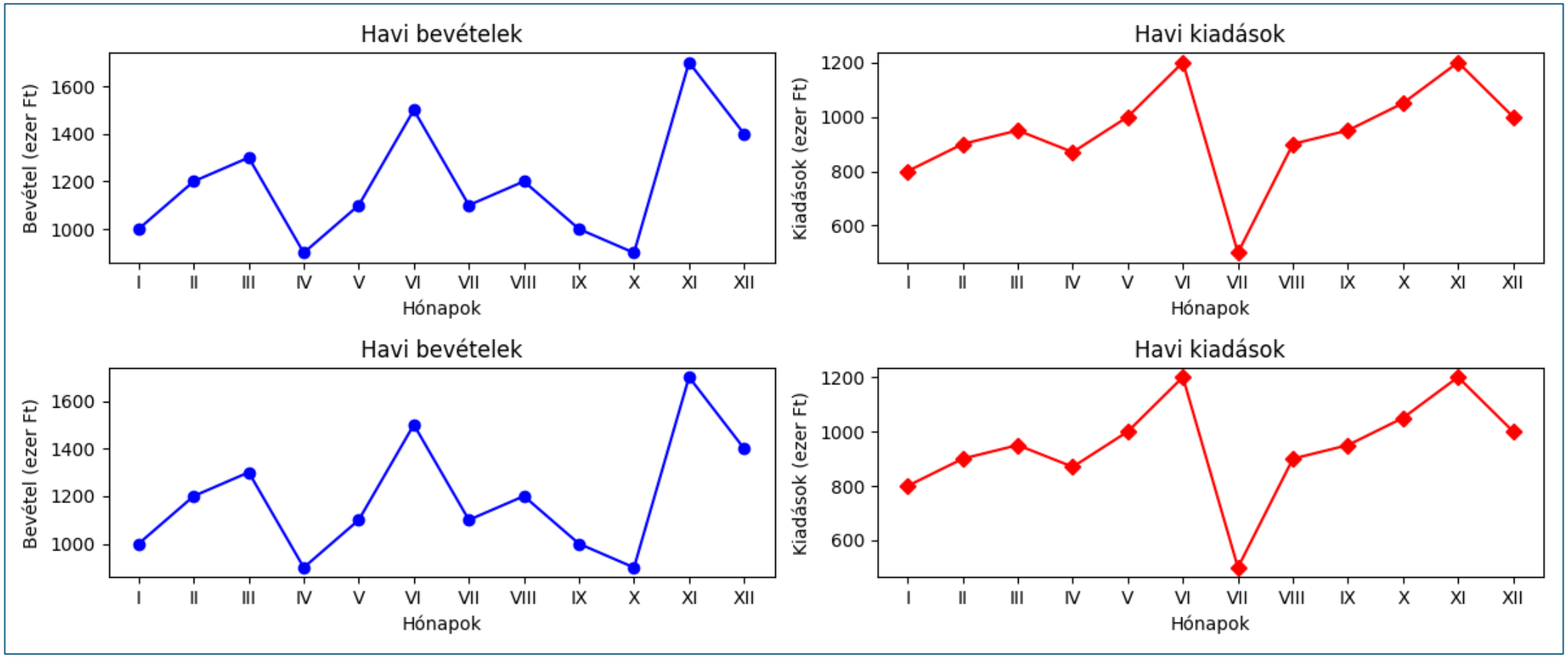
2. példa: Ábrázoljunk **egyszerre két diagramot** – a bevételeket és a kiadásokat.

```
kiadások = [800, 900, 950, 870, 1000, 1200, 500, 900,
950, 1050, 1200, 1000]
plt.figure(figsize=(12, 5))
# Első subplot
plt.subplot(1, 2, 1)  # 1 sor, 2 oszlop, 1. subplot
plt.plot(hónapok, bevételek, color='blue', marker='o')
plt.title('Havi bevételek')
plt.xlabel('Hónapok')
plt.ylabel('Bevétel (ezer Ft)')
# Második subplot
plt.subplot(1, 2, 2)  # 1 sor, 2 oszlop, 2. subplot
plt.plot(hónapok, kiadások, color='red', marker='D')
plt.title('Havi kiadások')
plt.xlabel('Hónapok')
plt.ylabel('Kiadások (ezer Ft)')
plt.tight_layout()  # Igazítás a helyes megjelenítéshez
plt.show()
```

Több diagram ábrázolása egy ablakban: **plt.subplot()** függvény használata. Paraméterek: sorok, oszlopok száma és a subplot pozíciója: indexeléssel. **plt.tight_layout()** az ábrák közötti térköz optimalizálásához.



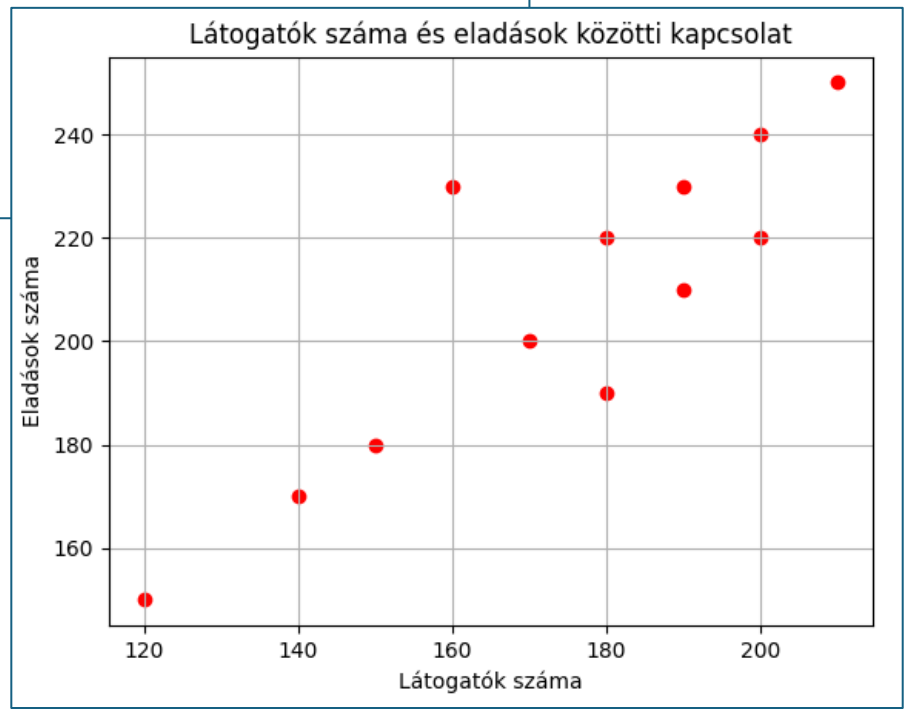
Feladat: Ábrázoljunk egyszerre négy diagramot - másolással



Scatter plotok a korrelációk vizsgálatához

3. példa: Látogatók száma és eladások közötti kapcsolat a **plt.scatter()** metódussal.

```
# Adatok generálása
látogatók = [120, 150, 180, 200, 170, 160, 140, 180, 190, 210, 190, 200]
eladások = [150, 180, 220, 240, 200, 230, 170, 190, 210, 250, 230, 220]
# Scatter plot készítése
plt.scatter(látogatók, eladások, color='red')
plt.title('Látogatók száma és eladások közötti kapcsolat')
plt.xlabel('Látogatók száma')
plt.ylabel('Eladások száma')
plt.grid(True)
plt.show()
```

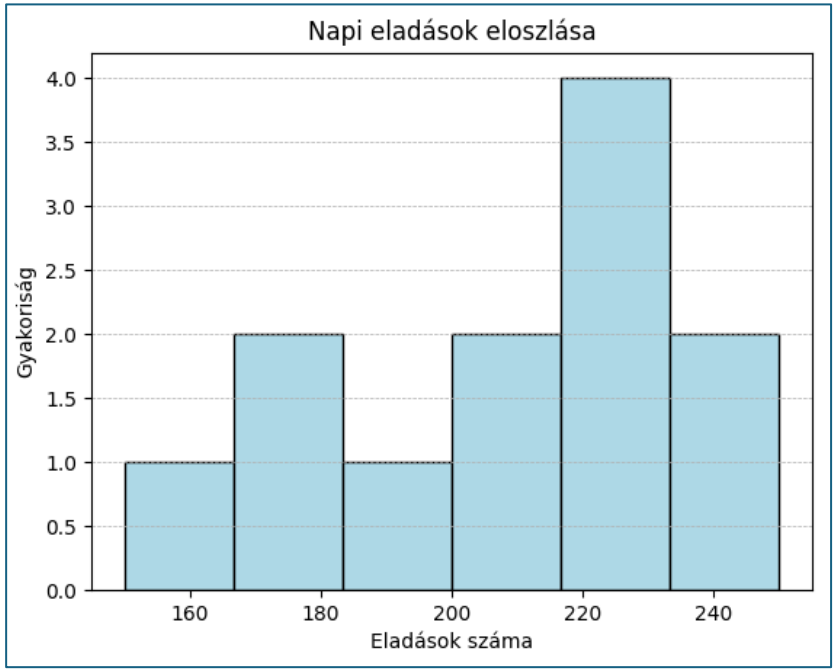


A **plt.grid(True)** jeleníti meg a rajzterület alapbeállítású vezetőrácsait.

Hisztogramok és eloszlásvizsgálatok

4. példa: Hisztogram az adatok gyakoriságának bemutatására a **plt.hist()** függvénnyel.

```
# Adatok generálása
eladások = [150, 180, 220, 240, 200, 230, 170, 190, 210, 250,
    230, 220]
# Hisztogram készítése
plt.hist(eladások, bins=6, color='lightblue', edgecolor='black')
plt.title('Napi eladások eloszlása')
plt.xlabel('Eladások száma')
plt.ylabel('Gyakoriság')
plt.grid(axis='y', linestyle='--', linewidth=0.5)
plt.show()
```

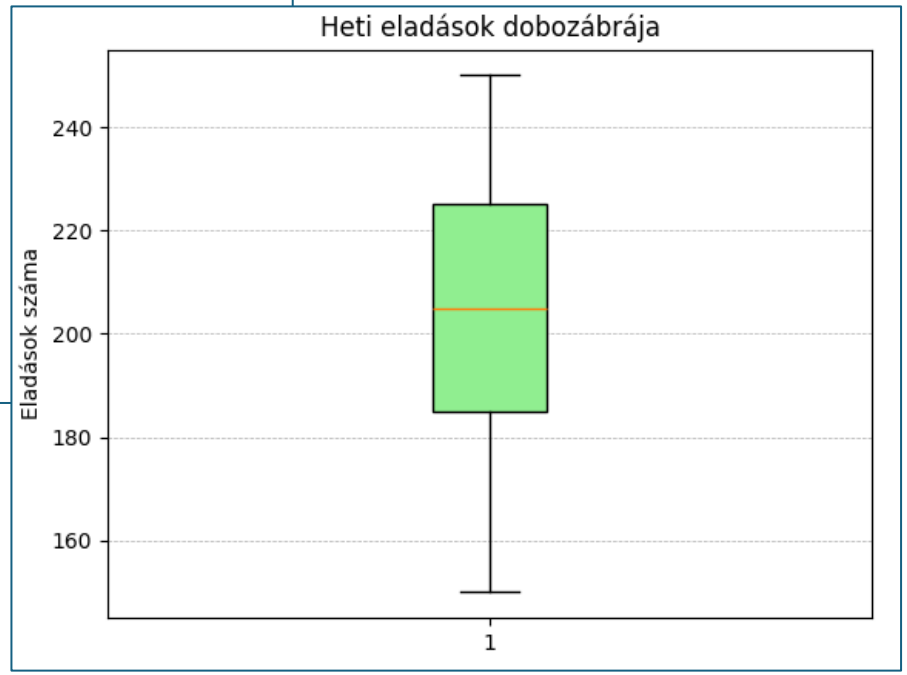


A **bins** paraméter határozza meg a sávok számát.
Az **edgecolor** az oszlopok körvonalának színét állítja be a jobb láthatóság érdekében.

Box plotok használata adatok szórásának bemutatására

5. példa: A dobozábrák vizuálisan mutatják az adatok szórását, középértékét, valamint a kiugró értékeket. A **plt.boxplot()** függvény hozza létre a dobozábrát.

```
Box plotok az adatok szórásának és kiugró értékeinek bemutatására.
# Heti eladások adatai
heti_eladasok = [150, 180, 220, 240, 200, 230,
   170, 190, 210, 250, 230, 220, 180, 195, 205]
# Box plot készítése
plt.boxplot(heti_eladasok, patch_artist=True,
    boxprops=dict(facecolor='lightgreen'))
plt.title('Heti eladások dobozábrája')
plt.ylabel('Eladások száma')
plt.grid(axis='y', linestyle='--', linewidth=0.5)
plt.show()
```



A **patch_artist** és **boxprops** paraméterek segítségével testreszabhatjuk a doboz megjelenését. Az ábra megmutatja a **mediánt**, a **kvartiliseket** és az esetleges kiugró értékeket.

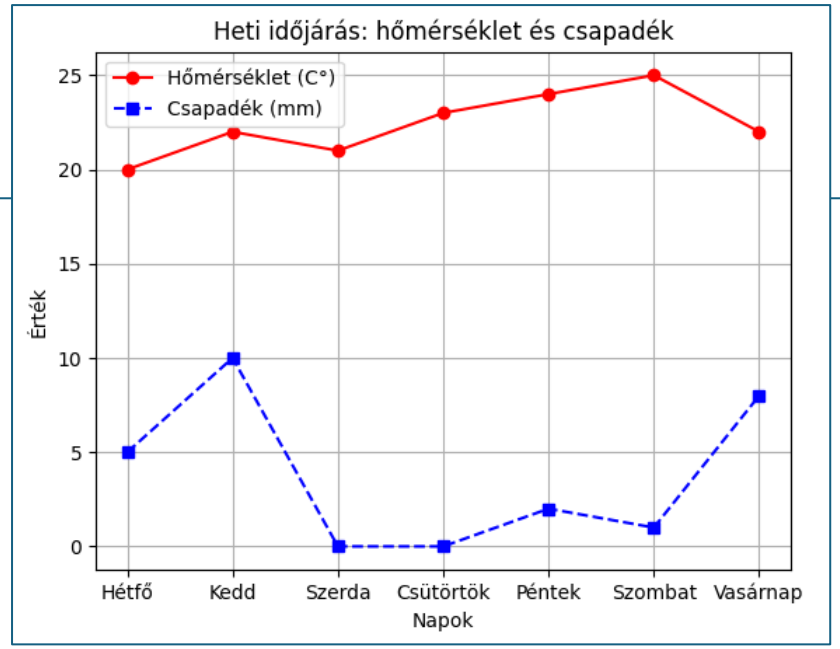
Mi a kód várható kimenete?

```
heti_eladasok_1 = [150, 180, 220, 240, 200, 230, 170, 190, 210, 250, 230, 220, 180, 195, 205]
heti_eladasok_2 = [160, 170, 210, 230, 190, 220, 180, 200, 220, 240, 210, 200, 170, 185, 195]

fig, ax = plt.subplots()

box1 = ax.boxplot(heti_eladasok_1, positions=[1], patch_artist=True,
boxprops=dict(facecolor='lightgreen’))

box2 = ax.boxplot(heti_eladasok_2, positions=[2], patch_artist=True,
boxprops=dict(facecolor='lightblue’))

plt.title('Heti eladások dobozábrája')
plt.ylabel('Eladások száma')
plt.xticks([1, 2], ['Heti eladások 1', 'Heti eladások 2'])
plt.grid(axis='y', linestyle='--', linewidth=0.5)

plt.show()
```

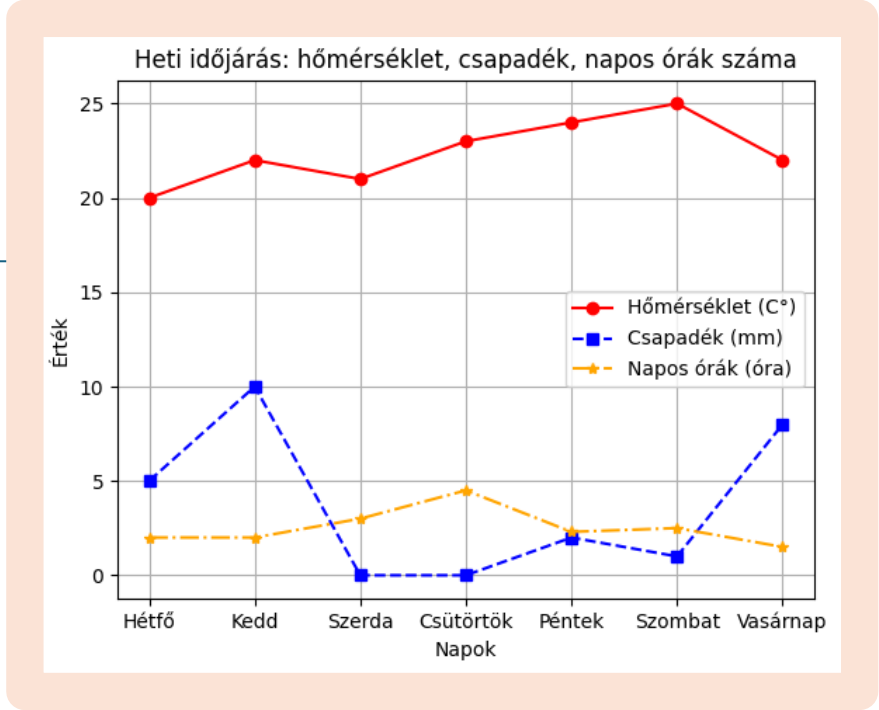
Kombinált diagramok 1. – Overlay diagram készítése

6. példa: Több különböző **adatsor** ábrázolása **egy diagramon**. A két grafikon egy város napi átlaghőmérsékletét és csapadékmennyiségét mutatja.

```
napok = ['Hétfő', 'Kedd', 'Szerda', 'Csütörtök', 'Péntek', 'Szombat', 'Vasárnap']
homerseklet = [20, 22, 21, 23, 24, 25, 22]  # Celsius fok
csapadek = [5, 10, 0, 0, 2, 1, 8]  # mm
plt.plot(napok, homerseklet, marker='o', color='red', label='Hőmérséklet (C°)')
plt.plot(napok, csapadek, marker='s', color='blue', linestyle='--', label='Csapadék (mm)')
plt.title('Heti időjárás: hőmérséklet és csapadék')
plt.xlabel('Napok')
plt.ylabel('Érték')
plt.legend()
plt.grid(True)
plt.show()
```



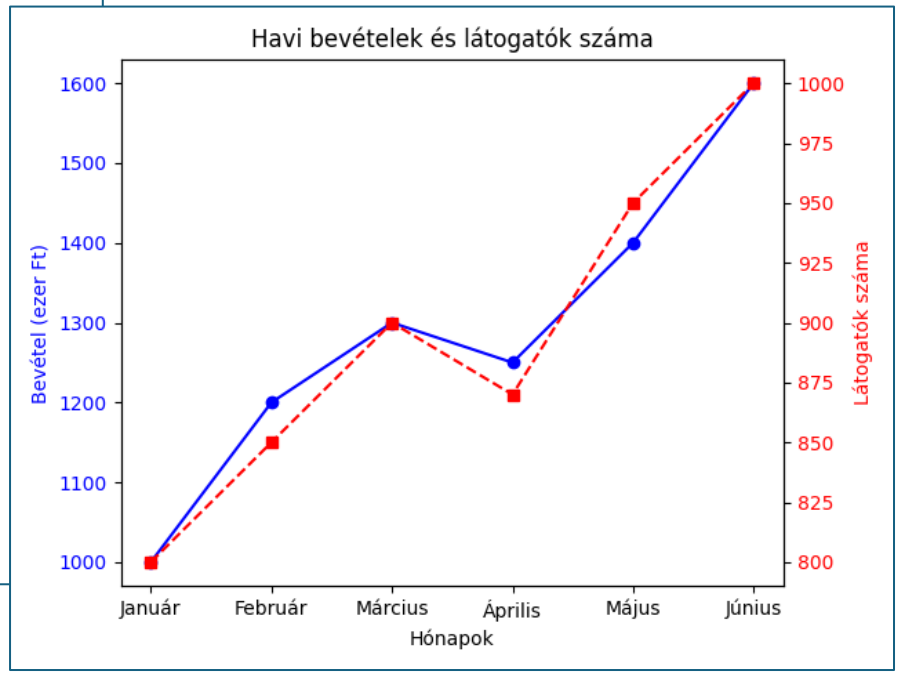
Napos órák adat-sor ábrázolása:



Kombinált diagramok 2. – Két y tengely egy diagramon

```
# Adatok generálása
honapok = ['Január', 'Február', 'Március', 'Április', 'Május', 'Június']
bevetelek = [1000, 1200, 1300, 1250, 1400, 1600]
latogatok = [800, 850, 900, 870, 950, 1000]
# Kombinált diagram készítése
fig, ax1 = plt.subplots()     #létrehoz egy ábrát és egy ax1 tengelyt
# Első y tengely: bevételek
ax1.plot(honapok, bevetelek, color='blue', marker='o', label='Bevétel')
ax1.set_xlabel('Hónapok')
ax1.set_ylabel('Bevétel (ezer Ft)', color='blue')
ax1.tick_params(axis='y', labelcolor='blue')
# Második y tengely: látogatók száma
ax2 = ax1.twinx()
ax2.plot(honapok, latogatok, color='red', marker='s', linestyle='--',
label='Látogatók száma')
ax2.set_ylabel('Látogatók száma', color='red')
ax2.tick_params(axis='y', labelcolor='red')
plt.title('Havi bevételek és látogatók száma')
fig.tight_layout()
plt.show()
```

7. példa: Egy bolt havi bevételeit és látogatóinak számát egy grafikonon jelenítjük meg a **plt.subplots()** és a **twinx()** metódusokkal.



2. Adatvizualizáció a Pandas eszközeivel

- A Pandas és a Matplotlib jól integráltak, ez lehetővé teszi a DataFrame objektumok közvetlen vizualizálását.
- A Matplotlib alapértelmezés szerint megjelenik, ha a **Pandas plot() metódusát** meghívjuk a **DataFrame-re**.
- Fontos csomagok importálása: **import pandas as pd**

  import matplotlib.pyplot as plt

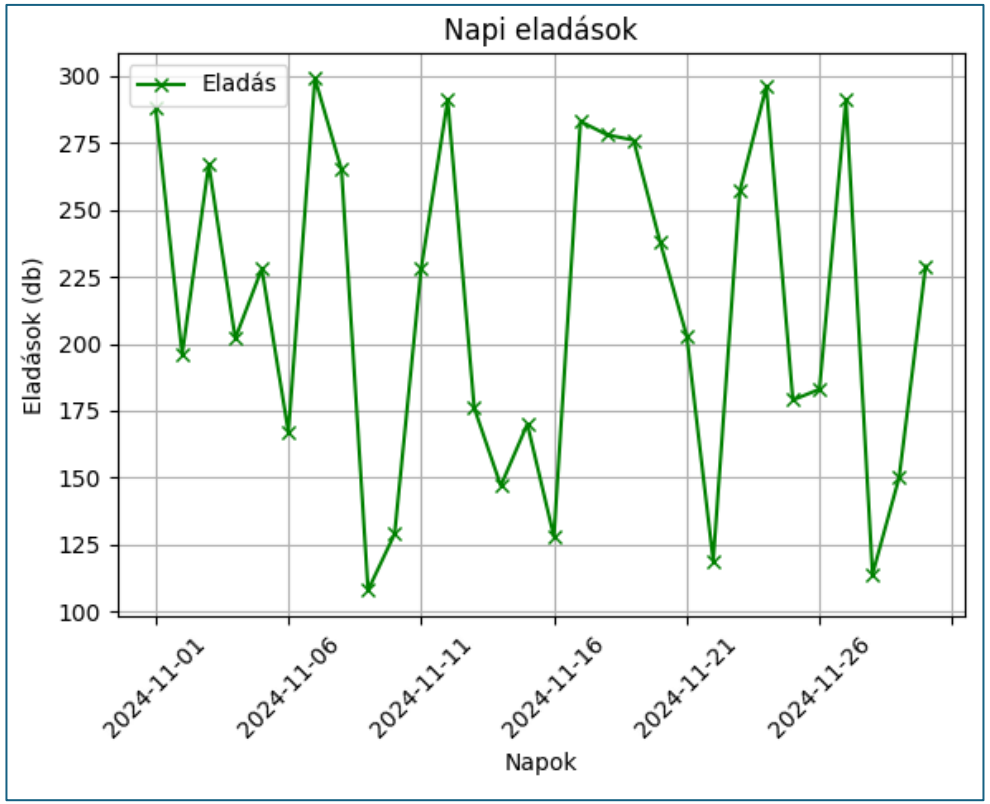
Beépített Pandas plot metódusok

A Pandas DataFrame és Series objektumai beépített **plot()** metódussal rendelkeznek.

8. példa: Vonal-diagram készítése közvetlenül Pandas segítségével

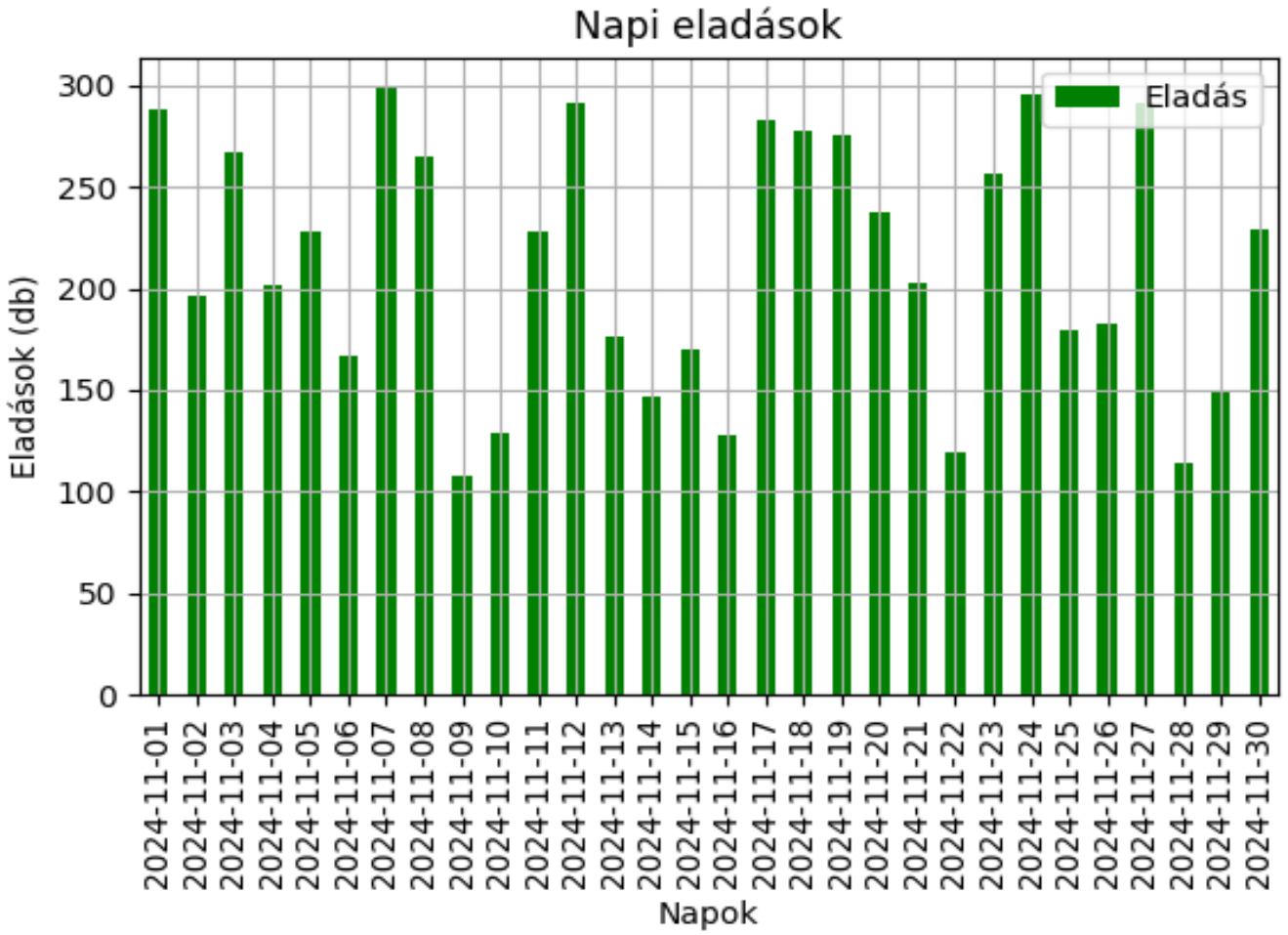
```
import pandas as pd
# CSV adatállomány betöltése
adatok = pd.read_csv('eladasok_30.csv')
adatok.plot(x='Nap', y='Eladás', kind='line', marker='x',
        color='green', title='Napi eladások')
plt.xlabel('Napok')
plt.ylabel('Eladások (db)')
plt.grid(True)
plt.show()
# Adatok megtekintése
print(adatok.head())
```

	Nap	Eladás	Látogatók
0	2024-11-01	288	78
1	2024-11-02	196	56
2	2024-11-03	267	75
3	2024-11-04	202	61
4	2024-11-05	228	70



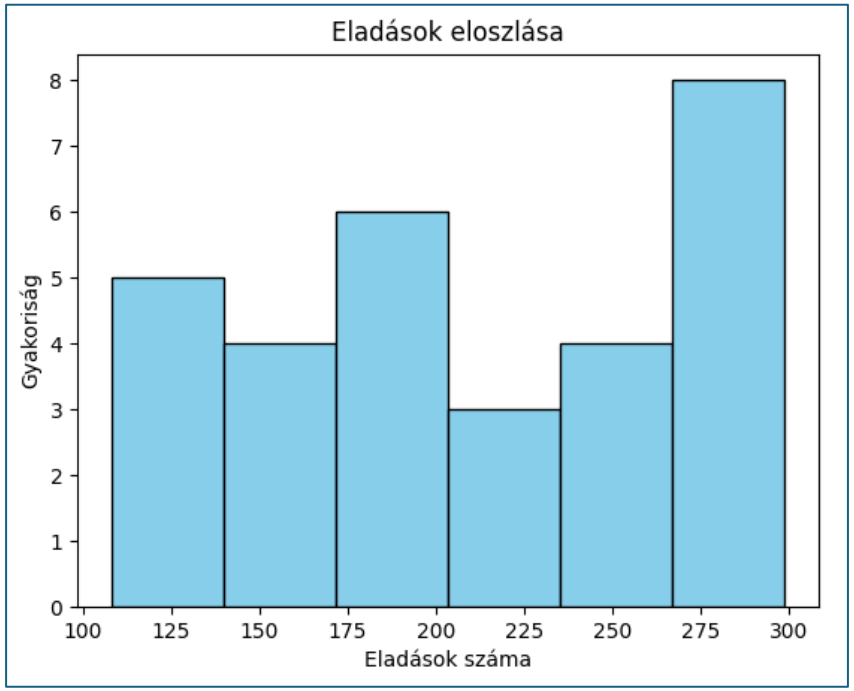
A **kind** paraméter határozza a meg a diagram típusát (pl. **line, bar**), az **x és y** a tengelyek megadása, a **marker** és **color** paraméterek a megjelenést szabályozzák.

Oszlopokkal: adatok.plot(x='Nap', y='Eladás', kind='bar', color='green', title='Napi eladások')



9. példa: Hisztogram készítése az eladások eloszlásának bemutatására

```
# Hisztogram készítése
adatok['Eladás'].plot(kind='hist', bins=6, color='skyblue', edgecolor='black',
title='Eladások eloszlása')
plt.xlabel('Eladások száma')
plt.ylabel('Gyakoriság')
plt.show()
```



A **kind='hist'** beállítással hisztogramot készítünk, amely az adatok eloszlását mutatja.

A **bins** paraméter határozza meg a csoportok számát.

3. A Seaborn könyvtár bevezetése és használata

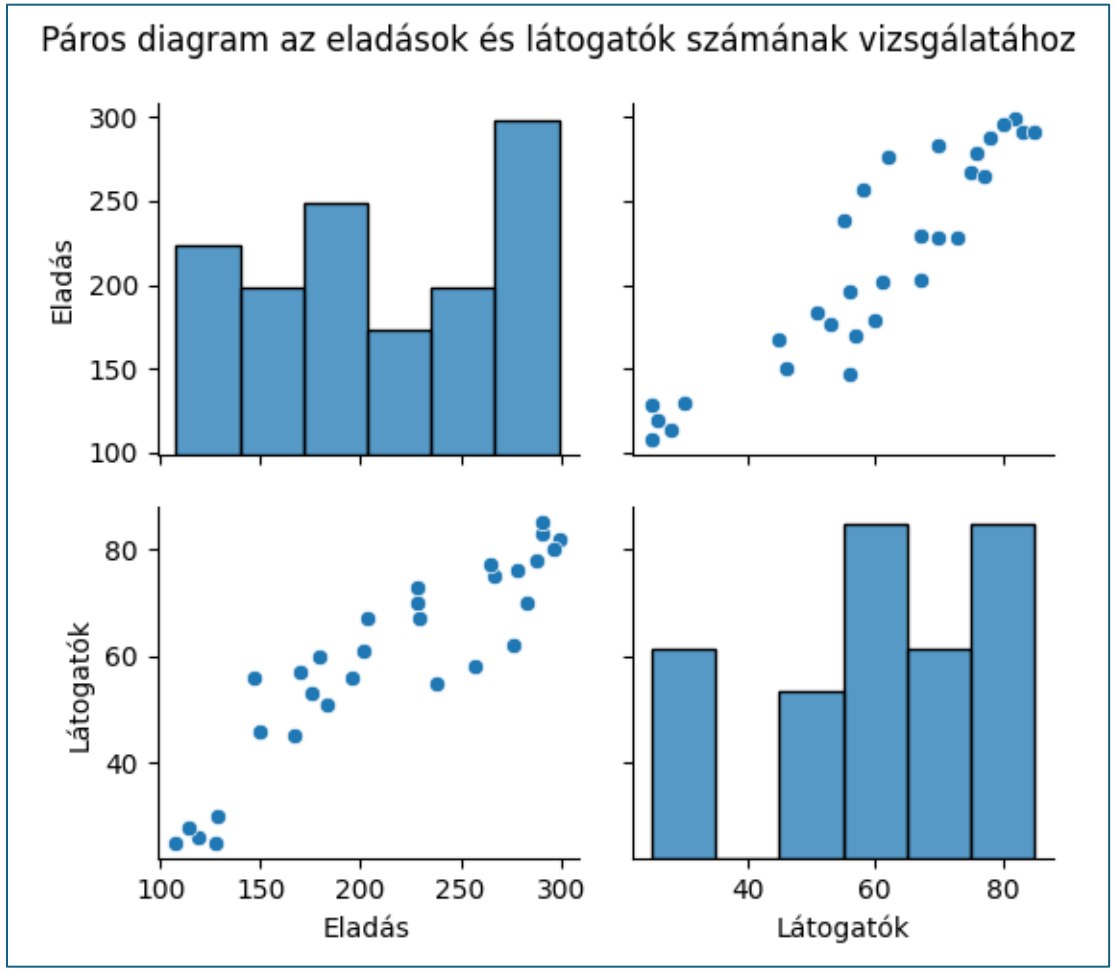
- A Seaborn egy magasabb szintű Python-könyvtár Matplotlib alapokon
- Előnyei: egyszerű használat, különleges diagramok, Pandas integráció
- Jól alkalmazható statisztikai adatok vizualizálására és összetett grafikonok létrehozására
- Hőtérképek, eloszlásdiagramok és trenddiagramok
- Beépített esztétikai beállítások
- Könnyen integrálható Pandas DataFrame-ekkel
- Több beépített plot típust tartalmaz, amelyekkel különböző típusú adatvizualizációkat készíthetünk.
- Telepítés: **pip install seaborn**

Seaborn pairplot()

A **pairplot()** több változó közötti kapcsolatok és eloszlások bemutatására szolgál, és automatikusan elkészíti a scatter plotokat és hisztogramokat.

10. példa: Többváltozós elemzés:

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
# Pairplot használata
sns.pairplot(adatok)
plt.suptitle('Páros diagram az eladások
és látogatók számának vizsgálatához', y=0.98)
plt.show()
```



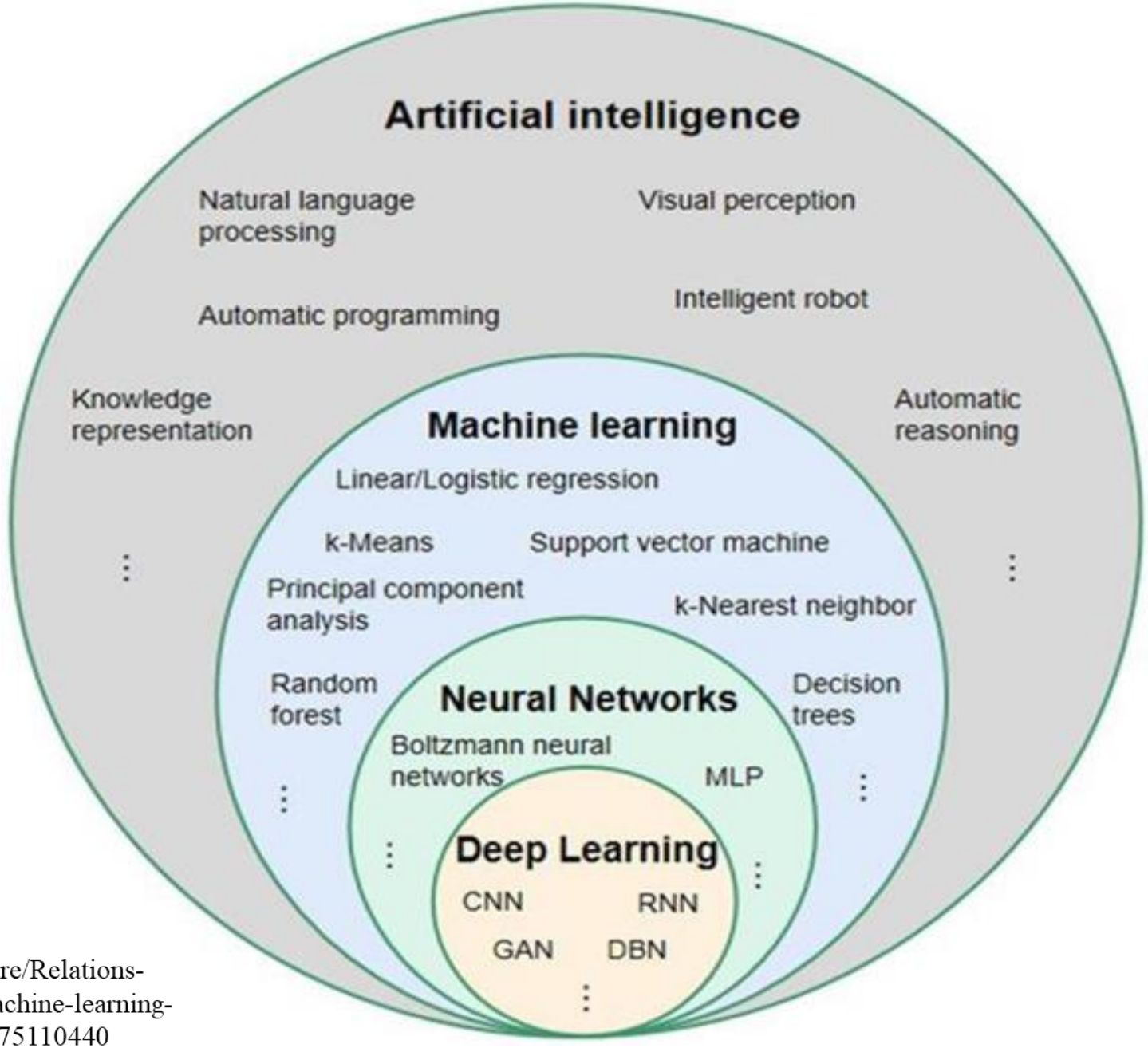
Mesterséges Intelligencia bevezetés

Az MI rendszerek jelenlegi képességei:

- **Problémákat megoldani** (adatelemzések, programkód írás, …),
- **Tanulni és alkalmazkodni** (gépi tanulási algoritmusok segítségével),
- **Döntéseket hozni** (önvezető autók, automatikus hitelbírálat, …).

Fogalmak

- **Mesterséges intelligencia – Artificial Intelligence (AI):** Az informatika azon ága, ami minden olyan technológiát lefed, amely emberi intelligenciát próbál imitálni.
- **Gépi tanulás - Machine Learning (ML):** Az MI azon ága, amely adatokból tanul.
- **Mély tanulás - Deep Learning (DL):** A gépi tanulás neurális hálózatokon alapuló formája.



https://www.researchgate.net/figure/Relations-between-artificial-intelligence-machine-learning-neural-network-and-deep_fig2_375110440

Machine Learning
A Scikit-learn könyvtár alkalmazása

A **Scikit-learn** Python egyik **gépi tanulási könyvtára:**

- Moduláris eszközkészletet biztosít gépi tanulási algoritmusokhoz, adatfeldolgozáshoz és modellértékeléshez.
- Beépített adathalmazokat is kínál, például az **Iris adatállományt**.
- **Főbb funkciói:** adatelőkészítés, modellek, algoritmusok, modellértékelés, ...
- **Telepítése:** `pip install scikit-learn`

A Scikit-learn könyvtár előnyei:

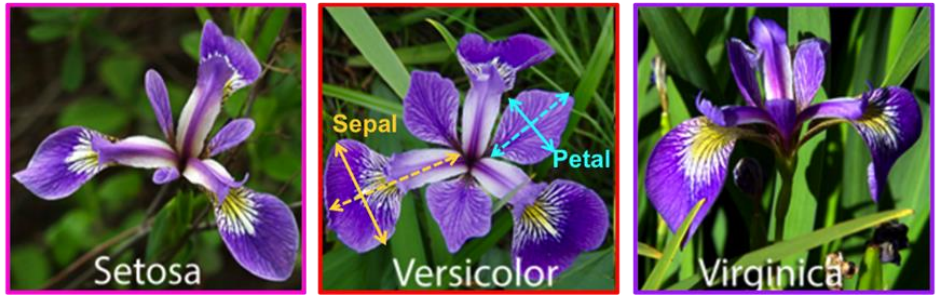
- Jól dokumentált.
- Egyszerűen használható API, amely gyors prototípus-fejlesztést tesz lehetővé.
- Számos gépi tanulási algoritmust kínál: osztályozás, regresszió, klaszterezés, …
- Könnyen használható együtt a NumPy, Pandas és Matplotlib könyvtárakkal.
- Nagy méretű adathalmaz esetén is hatékony

1. példa: Az Iris adatállomány vizsgálata

A scikit-learn könyvtár **load_iris** függvénye az Iris adatbázist tölti be. Az Iris adathalmaz három különböző íriszvirágfaj (Iris setosa, Iris versicolor, Iris virginica) mintáit tartalmazza, és minden minta négy jellemzővel rendelkezik. Az adathalmaz célja, hogy a virágfajokat a jellemzők alapján osztályozzuk.

Az Iris adathalmaz jellemzői (centiméterben):

Sepal length (cm): Csészelevél hossza.
Sepal width (cm): Csészelevél szélessége.
Petal length (cm): Sziromlevél hossza.
Petal width (cm): Sziromlevél szélessége.



Célváltozó (Target):

A célváltozó három különböző osztályt tartalmaz, az íriszvirágok fajait:

0: Iris setosa 1: Iris versicolor 2: Iris virginica

Az alkalmazott modulok összefoglalva:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

Funkciók:

- Adatok betöltése egy DataFrame-be
- Adatelőkészítés: hiányzó értékek kezelése, skálázás az ML modell igényei szerint, típusok beállítása (pl. int, datetime).
- ML algoritmus kiválasztása: lineáris regresszió, döntési fa, SVM, ...
- Modellépítés: tanító és teszt (train-test) adathalmazok szétválasztása, az algoritmus betanítása, eredmények megjelenítése.
- Modellértékelés: metriák és validációs eszközök alapján (pontosság).

1. Adatok betöltése az Iris-ből és az adatok előkészítése

```
from sklearn.datasets import load_iris

iris_adatok = load_iris()                # Adatok betöltése dict-be
print(iris_adatok.keys())                # Kulcsok listázása
print(iris_adatok.target_names)          # Célváltozók nevei
#print(iris_adatok.data)                 # Adatok listázása
#print(iris_adatok.DESCR)                # Az Iris adatállomány rövid leírása
adatok = pd.DataFrame(iris_adatok.data, columns=iris_adatok.feature_names)
célváltozó = pd.Series(iris_adatok.target, name="cél")

print(adatok.head())                     # adatok dataframe megtekintése
```

```
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
['setosa' 'versicolor' 'virginica']
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

2. Példa az adatok skálázására a StandardScaler modullal

```
from sklearn.preprocessing import StandardScaler
skala = StandardScaler()                          # Skálázó inicializálása
adatok_skálázva = skala.fit_transform(adatok)   # A fit_transform metódussal skálázunk
# Skálázott adatok visszaalakítása DataFrame-be
adatok_skálázva_df = pd.DataFrame(adatok_skálázva, columns=adatok.columns)
print(adatok_skálázva_df.head())
```

Ez a kódrészlet a scikit-learn könyvtár **StandardScaler osztályát** használja az adatok skálázására. Az adatok különböző jellemzőit azonos skálára hozzuk, ez szükséges a gépi tanulási algoritmusok hatékony működéséhez. A StandardScaler az adatokat úgy skálázza, hogy azok **átlagát 0-ra**, **szórását pedig 1-re** állítja.

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	-0.900681	1.019004	-1.340227	-1.315444
1	-1.143017	-0.131979	-1.340227	-1.315444
2	-1.385353	0.328414	-1.397064	-1.315444
3	-1.506521	0.098217	-1.283389	-1.315444
4	-1.021849	1.249201	-1.340227	-1.315444

3. Lineáris regresszió ML algoritmus alkalmazása az adatok dataframe-re

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Adatfelosztás: tanító és teszt halmazra (arány: 80% és 20%)
x_tanító, x_teszt, y_tanító, y_teszt = train_test_split(adatok, célváltozó, test_size=0.2,
random_state=42)

# Modell inicializálása és betanítása
linearis_model = LinearRegression()
linearis_model.fit(x_tanító, y_tanító)

# Predikció
predikciók = linearis_model.predict(x_teszt)            # A predict metódus alkalmazása
print("Első 5 predikció:", predikciók[:5])
print("Valós értékek:", y_teszt[:5].values)

# Pontosság kiszámítása a teljes teszt adatra
pontossag = linearis_model.score(x_teszt, y_teszt)     # Pontosság:  score metódus
print("Pontosság:", pontossag)
```

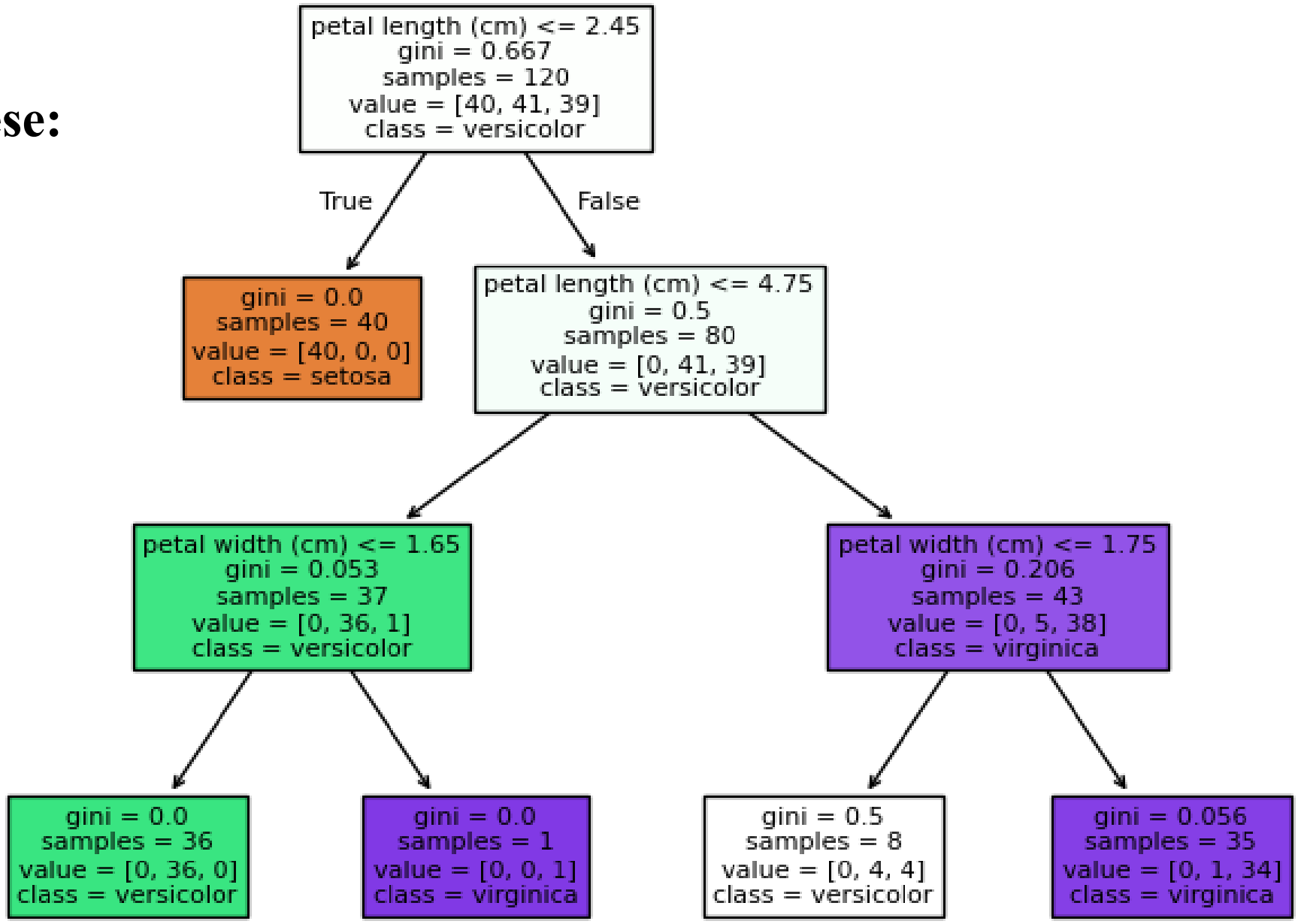
```
Első 5 predikció: [ 1.23071715   -0.04010441   2.21970287    1.34966889    1.28429336]
Valós értékek: [1  0  2  1  1]
Pontosság: 0.9468960016420045
```

4. Döntési fa ML algoritmus alkalmazása az adatok dataframe-re

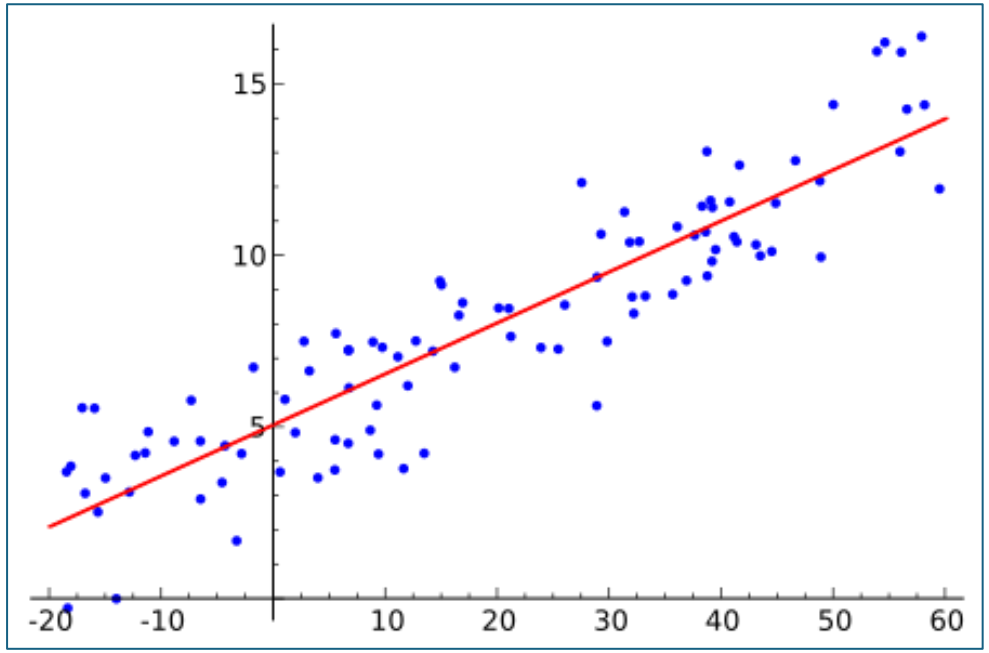
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Modell inicializálása és betanítása
dontesi_fa = DecisionTreeClassifier(max_depth=3, random_state=42)
dontesi_fa.fit(x_tanító, y_tanító)

# Predikció és pontosság értékelése
predikciók_fa = dontesi_fa.predict(x_teszt)
print("Első 5 predikció:", predikciók_fa[:5])
print("Valós értékek:", y_teszt[:5].values)
pontosság = accuracy_score(y_teszt, predikciók_fa)
print("Döntési fa pontossága:", pontosság)

#Megjelenítés
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
plot_tree(dontesi_fa, feature_names=iris_adatok.feature_names,
class_names=iris_adatok.target_names, filled=True)
plt.show()
```

```
Első 5 predikció: [1 0 2 1 1]
Valós értékek: [1 0 2 1 1]
Döntési fa pontossága: 1.0
```

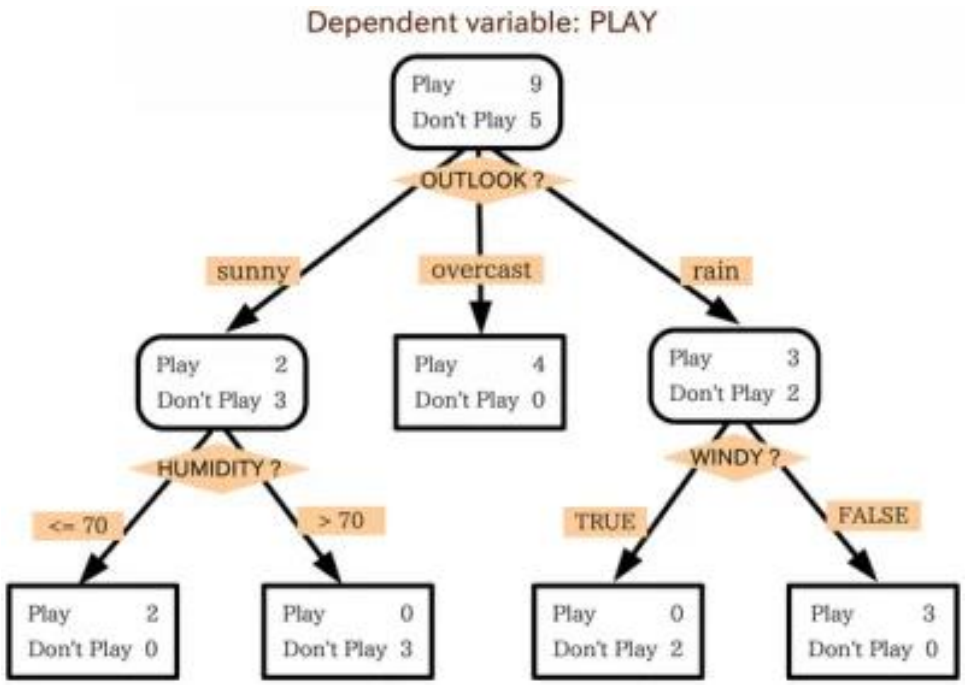
Döntési fa megjelenítése:



A lineáris regresszió és a döntési fa jellemzői



https://hu.wikipedia.org



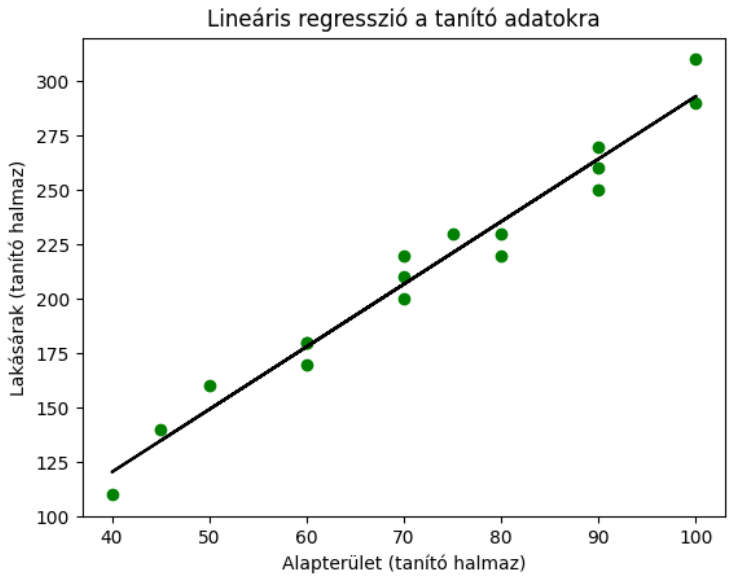
https://www.unite.ai/hu/what-is-a-decision-tree/

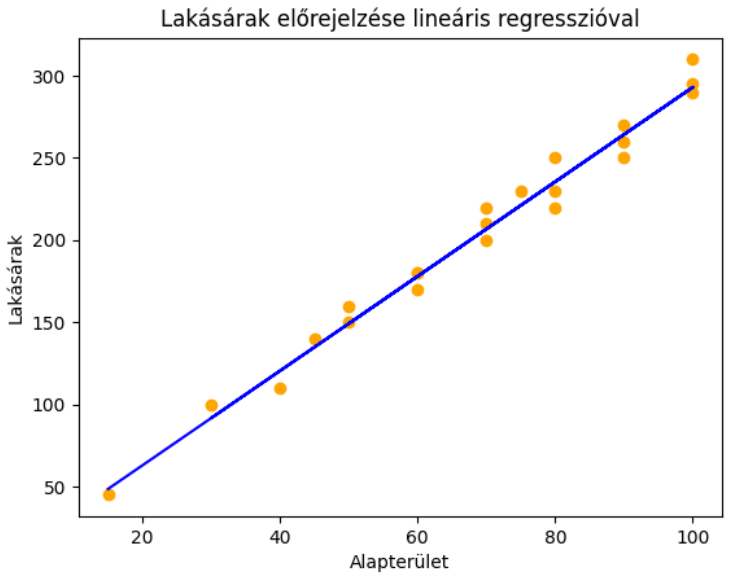
1. Lineáris regresszió példa: Lakásárak előrejelzése az alapterület alapján

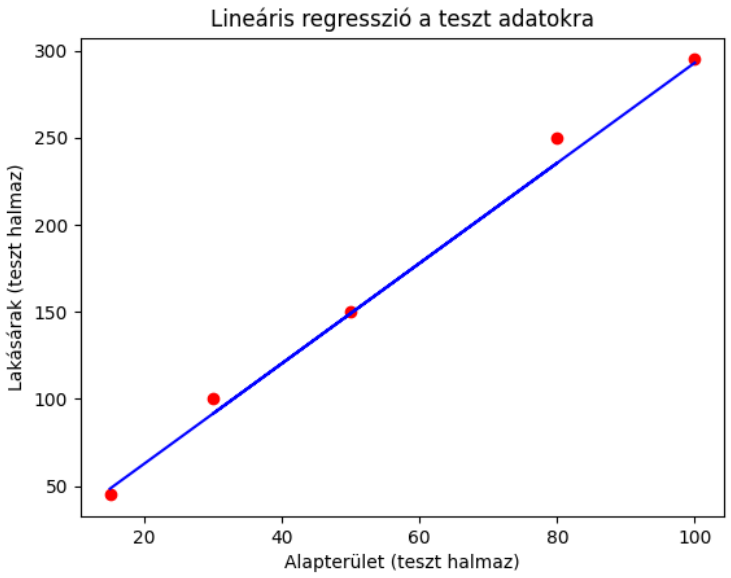
```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
adatok = pd.DataFrame({
   "alapterulet": [15, 50, 60, 70, 80, 90, 100, 75, 30, 40, 45, 60, 70, 80, 90, 100, 50, 60, 70, 80, 90, 100],
   "ar":[45, 150, 180, 200, 220, 250, 290, 230, 100, 110, 140, 170, 220, 250, 270, 295, 160, 180, 210, 230, 260, 310]
})
x = adatok[["alapterulet"]]                    # Független és függő változók
y = adatok["ar"]
x_tanító, x_teszt, y_tanító, y_teszt = train_test_split(x, y, test_size=0.2, random_state=42)
linearis_model = LinearRegression()        # Lineáris regresszió inicializálása és betanítása
linearis_model.fit(x_tanító, y_tanító)
w = linearis_model.coef_[0]                    # Modell paraméterei
b = linearis_model.intercept_
print("Meredekség (w):", w)
print("Elfogási pont (b):", b)
print("Egyenes egyenlete: Y = {:.2f} * X + {:.2f}".format(w, b))
predikciók = linearis_model.predict(x_teszt)              # Predikciók a teszt halmazra
print("Első 5 predikció:", predikciók[:5])
print("Valós értékek:", y_teszt[:5].values)
pontossag = linearis_model.score(x_teszt, y_teszt)
print("Modell pontossága:", pontossag)
```

```
Meredekség (w): 2.8769906644700716
Elfogási pont (b): 5.370675453047738
Egyenes egyenlete: Y = 2.88 * X + 5.37
Első 5 predikció: [ 48.52553542  235.52992861   91.68039539
149.22020868    293.0697419 ]
Valós értékek: [ 45 250 100 150 295]
Modell pontossága: 0.9931199124676021
```

```
# Teszt adatok és a lineáris regresszió megjelenítése
import matplotlib.pyplot as plt
plt.scatter(x_teszt, y_teszt, color='red', label='Tanító adatok')
plt.plot(x_teszt, linearis_model.predict(x_teszt), color='blue', label='Lineáris regresszió')
plt.xlabel('Alapterület (teszt halmaz)')
plt.ylabel('Lakásárak (teszt halmaz)')
plt.title('Lineáris regresszió a teszt adatokra')
plt.show()
```







2. Döntési fa jellemzői

- A döntési fa egy **gépi tanulási algoritmus.**
- A döntési fa hierarchikus döntéseket hoz, amelyek egy fastruktúrába vannak szervezve.
- A fa alakú diagramban minden **csomópont** egy változó egy lehetséges értékét képviseli, minden **ág** egy döntési szabályt jelöl, és minden **levél** az előrejelzés eredményét adja meg.

Döntési szabályok kiválasztása a következő kritériumok egyikével:

1. **Gini index:** az adathalmaz homogenitásának mérőszáma.

   Cél: **minimalizálni** a Gini indexet a szétválasztások után.

$$Gini = 1 - \sum_{i=1}^{C} (p_i)^2$$

p_i az adott osztály valószínűsége.

2. **Entrópia és IG:**

   Entrópia : A rendezetlenség mértéke.

   Cél: **csökkenteni az entrópiát** a szétválasztások után.

   Információnyereség (Information Gain):

   Az entrópia változása egy adott osztályozás előtt és után.

   Cél: maximalizálni az információnyereséget.

$$Entropy = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

p_i az adott osztály valószínűsége.

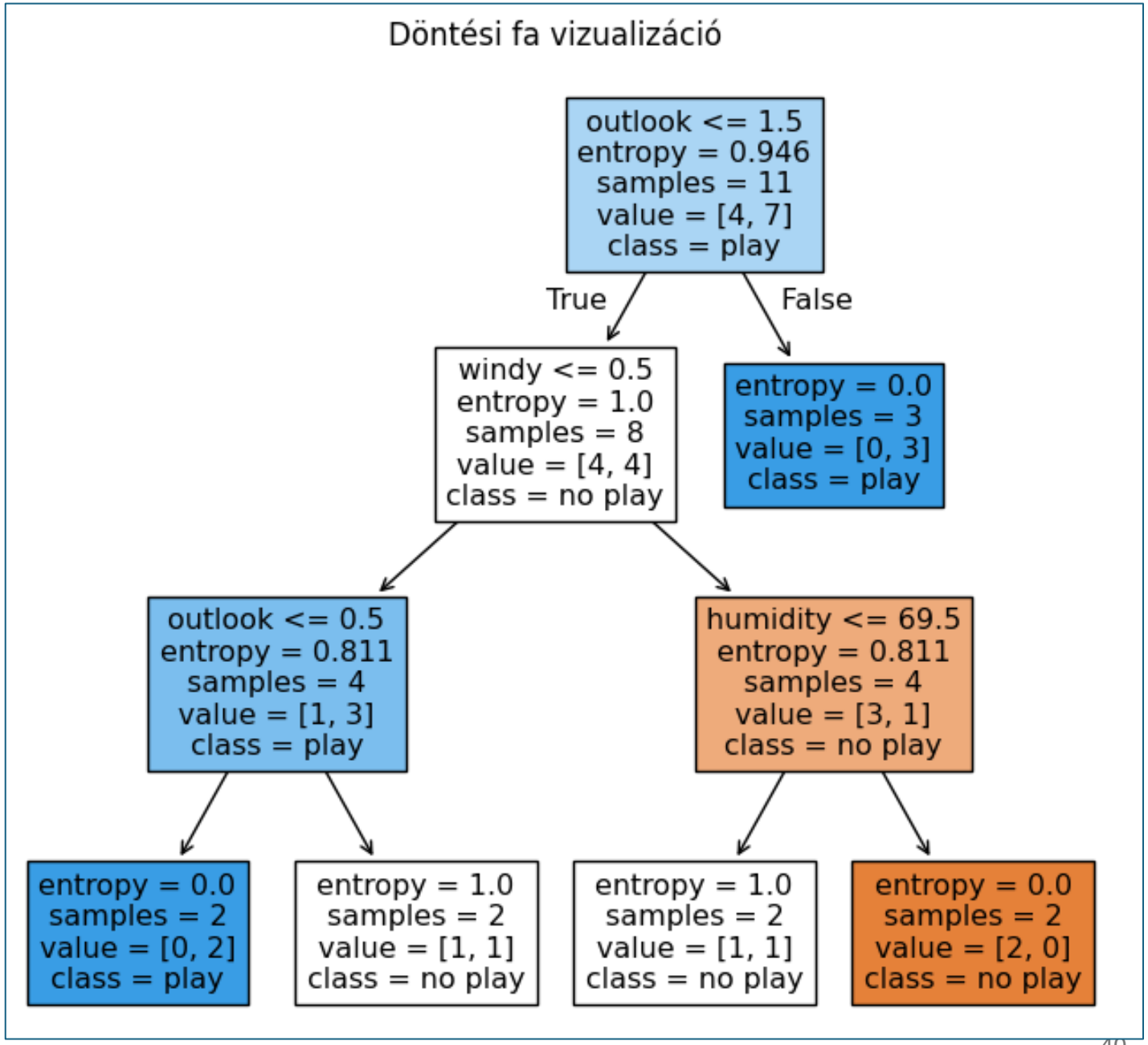
3. példa: Tenisz játék és az időjárás döntési fával

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

adatok = pd.read_csv('Tenisz_csv_uj.csv')          # Forrásfájl beolvasása
print(adatok.head())
célváltozó = adatok['play’]          # Célváltozó kiválasztása
jellemzők = adatok.drop(columns=['play'])
# Adatfelosztás: tanító és teszt halmaz
x_tanító, x_teszt, y_tanító, y_teszt = train_test_split(jellemzők, célváltozó, test_size=0.2,
random_state=42)

# Modell inicializálása és betanítása
fa_modell = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
fa_modell.fit(x_tanító, y_tanító)
predikciók_fa = fa_modell.predict(x_teszt)      # Predikció
print("Első 5 predikció:", predikciók_fa[:5])
print("Valós értékek:", y_teszt[:5].values)
pontossag = accuracy_score(y_teszt, predikciók_fa)        # Pontosság kiszámítása
print("Pontosság:", pontossag)

plt.figure(figsize=(8, 6))                    # Döntési fa megjelenítése
plot_tree(fa_modell, feature_names=jellemzők.columns, class_names=['no play','play'], filled=True)
plt.title("Döntési fa vizualizáció")
plt.show()
```

Oszlopnevek és értékek:

outlook: 2 – overcast
1 – sunny
0 – rain
humidity: integer
windy: 0 – False
1 – True
play: 0 – no play
1 – play

	outlook	humidity	windy	play
0	2	67	1	1
1	2	75	1	1
2	2	67	0	1
3	2	75	0	1
4	1	67	1	1



Felügyelt és nem felügyelt gépi tanulási algoritmusok az iris-re

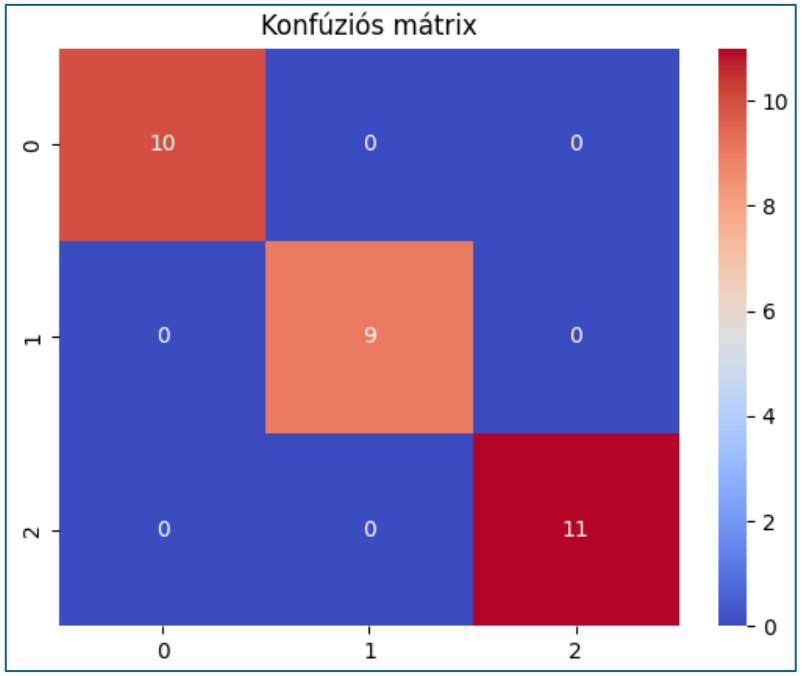
1. Felügyelt tanulás (Supervised Learning) példa: Logistic Regression

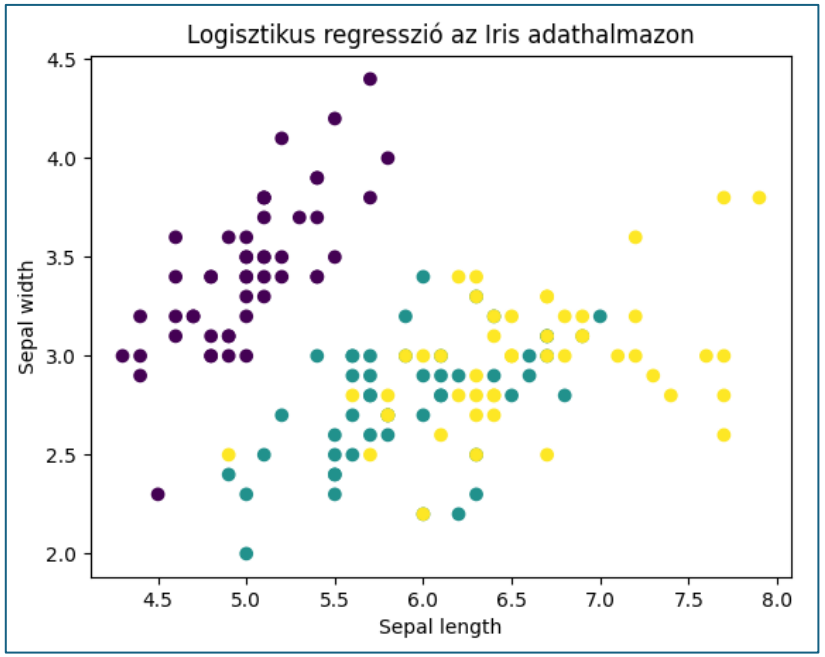
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
# Adatok betöltése
iris = load_iris()
X = iris.data
y = iris.target
# Adatok felosztása tanító és teszt halmazokra
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Modell betanítása
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
# Előrejelzések készítése
y_pred = model.predict(X_test)
# Pontosság kiszámításaPontosság: 1.0
accuracy = accuracy_score(y_test, y_pred)
print("Pontosság:", accuracy)
```

Pontosság: 1.0

```
# Konfúziós mátrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("Konfúziós mátrix:")
print(conf_matrix)
# Konfúziós mátrix megjelenítése heatmap-pel
import seaborn as sns
sns.heatmap(conf_matrix, annot=True,
cmap='coolwarm')
plt.title('Konfúziós mátrix')
plt.show()
#Logisztikus regresszió megjelenítése scatter-en
import matplotlib.pyplot as plt
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.title('Logisztikus regresszió az Iris adathalmazon')
plt.show()
```

Pontosság: 1.0
Konfúziós mátrix:
[[10 0 0]
[0 9 0]
[0 0 11]]





Konfúziós mátrix (Confusion matrix) felépítése:

A konfúziós mátrix egy négyzet alakú mátrix, amelynek sorai a valós osztályokat, oszlopai pedig a modell által előrejelzett osztályokat reprezentálják. Az egyes cellák számai azt mutatják, hogy hány adatpont tartozik az adott valós és előrejelzett osztály kombinációhoz. **Kétosztályos (bináris) osztályozás** esetén:

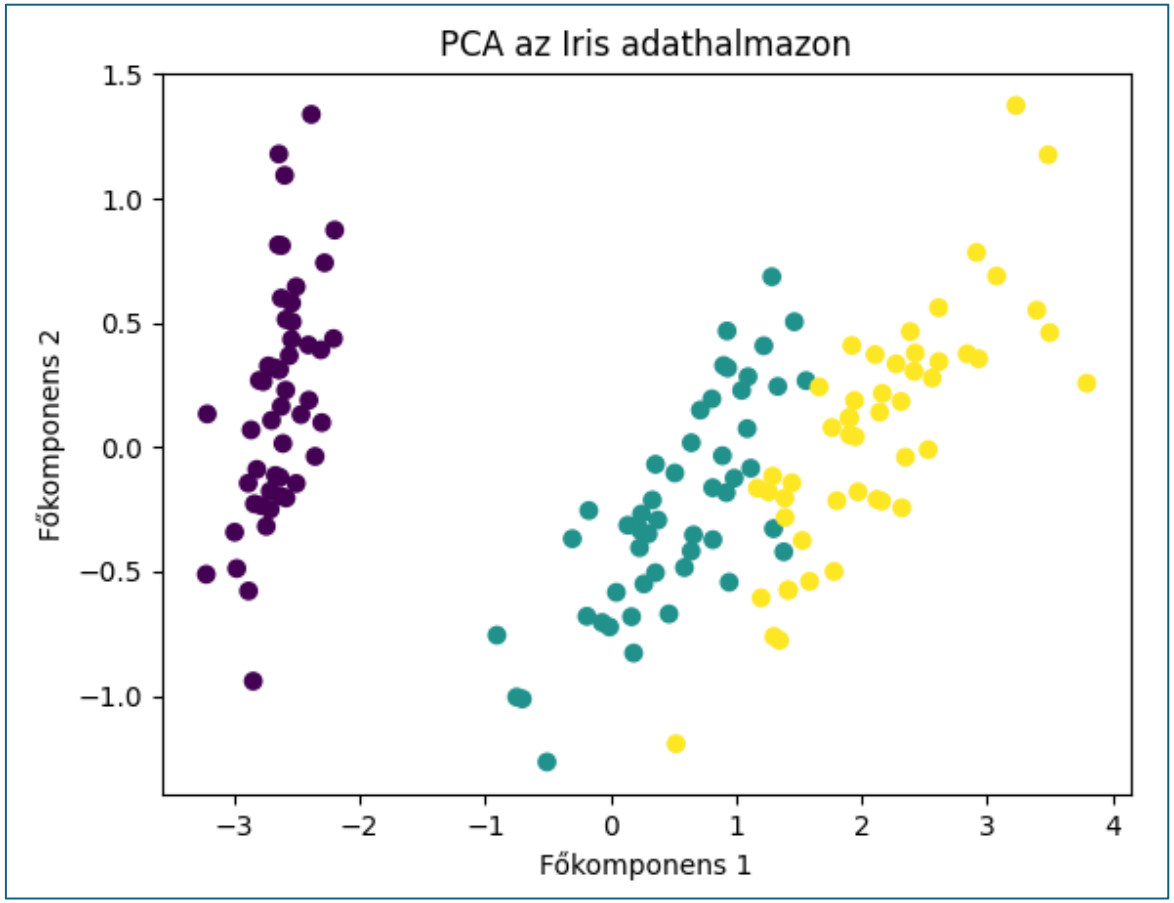
Valós \ Előrejelzett	Pozitív (1)	Negatív (0)
Pozitív (1)	TP	FN
Negatív (0)	FP	TN

- TP (True Positive): Helyesen pozitívnak előrejelzett esetek száma.
- TN (True Negative): Helyesen negatívnak előrejelzett esetek száma.
- FP (False Positive): Tévesen pozitívnak előrejelzett esetek száma (hamis riasztás).
- FN (False Negative): Tévesen negatívnak előrejelzett esetek száma (kihagyott riasztás).

A konfúziós mátrix segítségével számos teljesítménymutató számítható ki, például:
Pontosság (Accuracy): Az összes helyes előrejelzés aránya az összes előrejelzéshez képest
Precizitás (Precision): Az összes helyesen pozitívnak előrejelzett eset aránya az összes pozitívnak előrejelzett esethez képest.
Érzékenység (Recall) vagy True Positive Rate (TPR): Az összes helyesen pozitívnak előrejelzett eset aránya az összes valós pozitív esethez képest.
F1 pontszám (F1 Score): A precizitás és az érzékenység harmonikus átlaga.

2. Nem felügyelt tanulás (Unsupervised Learning) példa: PCA (főkomponens analízis)

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
# Adatok betöltése
iris = load_iris()
X = iris.data
# Dimenziócsökkentés PCA-val
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
# Adatok vizualizálása
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=iris.target)
plt.xlabel('Főkomponens 1')
plt.ylabel('Főkomponens 2')
plt.title('PCA az Iris adathalmazon')
plt.show()
```



A főkomponens-analízis (Principal Component Analysis, PCA) egy dimenziócsökkentési technika, amely az eredeti, magas dimenziójú adathalmazt egy alacsonyabb dimenziójú térbe vetíti úgy, hogy az adatok legfontosabb információit megőrizze.

A főkomponensek együtthatóinak megjelenítése: ezek megmutatják, hogy az eredeti jellemzők hogyan járulnak hozzá az egyes főkomponensekhez:

```
# Főkomponensek együtthatóinak megjelenítése
components = pd.DataFrame(pca.components_, columns=iris_adatok.feature_names,
index=['Főkomponens 1', 'Főkomponens 2'])
print(components)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
Főkomponens 1	0.361387	-0.084523	0.856671	0.358289
Főkomponens 2	0.656589	0.730161	-0.173373	-0.075481

Az első főkomponens leginkább a sziromlevél (petal) hosszát és szélességét jelzi, míg a második főkomponens főleg a csészelevél (sepal) hosszát és szélességét jelzi. Az együtthatók előjele és nagysága megmutatja, hogy az egyes jellemzők hogyan járulnak hozzá a főkomponensekhez.
A főkomponensek nem közvetlenül az eredeti jellemzők, hanem azok kombinációi, amelyek az adatok legfontosabb információit tartalmazzák.

Köszönöm a figyelmet!

Konzultáció

Minden héten csütörtökön 18:00 – 20:00